

AI QB UNIT 3 (4 marks)

1. Define problem solving in AI.

1. Problem solving in AI is the process of finding a solution for a given problem using logical steps. It helps intelligent agents achieve goals by exploring different possible actions.
2. In AI, a problem is usually represented using states, actions, initial state, and goal state. The AI system searches through these states to reach the final solution.
3. Problem solving methods use search algorithms to explore the search space efficiently. These algorithms can be uninformed or informed depending on available knowledge.
4. Uninformed search algorithms work without extra information about the goal state. Informed search algorithms use heuristics to guide the search toward the solution.
5. AI problem solving is widely used in robotics, games, navigation systems, and expert systems. It allows machines to make decisions and solve complex tasks automatically.
6. The main objective of problem solving in AI is to find the best and most efficient solution. It also aims to reduce time, cost, and search effort while solving problems.

2. What is Breadth First Search (BFS)?

1. Breadth First Search is a search algorithm used for traversing trees and graphs. It explores nodes level by level starting from the root node.
2. BFS first visits all nodes at the current depth before moving to the next level. It works in a breadthwise manner, which gives it the name Breadth First Search.
3. BFS uses a FIFO queue data structure for storing and processing nodes. The node added first into the queue is explored first.
4. This algorithm is complete because it can find a solution if one exists at a finite depth. It is also optimal when path cost increases with depth.
5. One advantage of BFS is that it can find the shortest path with minimum steps. However, it requires a large amount of memory to store all nodes of each level.
6. BFS is commonly used in shortest path problems, network traversal, and AI search problems. Its time and space complexity are both $O(b^d)$.

3. What is Depth Limited Search?

1. Depth Limited Search is a search algorithm similar to Depth First Search. It searches only up to a fixed depth limit in the search tree.
2. This algorithm prevents the infinite path problem that may occur in normal DFS. Nodes beyond the depth limit are not expanded further.
3. In Depth Limited Search, nodes at the maximum depth are treated as if they have no child nodes. This helps control unnecessary deep searching.
4. The algorithm can fail in two ways: standard failure and cut-off failure. Standard failure means no solution exists, while cut-off failure means no solution was found within the depth limit.
5. One major advantage of DLS is that it is memory efficient compared to BFS. However, it may not always find the optimal solution if multiple solutions exist.
6. The time complexity of DLS is $O(b\ell)$, where b is branching factor and ℓ is depth limit. Its space complexity is $O(b \times \ell)$.

4. Define Iterative Deepening DFS (DFID).

1. Iterative Deepening Depth First Search is a search algorithm that combines features of BFS and DFS. It repeatedly performs DFS with increasing depth limits.
2. The algorithm starts with a small depth limit and gradually increases it until the goal node is found. This helps in searching large spaces efficiently.
3. DFID combines the fast searching ability of BFS with the low memory usage of DFS. It is useful when the depth of the goal node is unknown.
4. The algorithm is complete if the branching factor is finite. It can also be optimal when path costs do not decrease with depth.
5. One disadvantage of DFID is that it repeats searches from previous iterations. Because of repeated work, some extra processing time is required.
6. The time complexity of DFID is $O(b^d)$, and its space complexity is also $O(b^d)$. It is widely used in AI problem-solving and game searching applications.

5. What is an informed search algorithm?

1. An informed search algorithm is a search technique that uses extra knowledge to find the goal state efficiently. This additional knowledge helps reduce the search space and improves performance.
2. Informed search algorithms use heuristic information to guide the search process. Because of this, they are also called heuristic search algorithms.
3. A heuristic function estimates how close a current state is to the goal state. It is represented by $h(n)$, where lower values usually mean the goal is nearer.
4. These algorithms are more useful for large and complex search spaces. They can often find solutions faster than uninformed search algorithms.
5. Informed search methods use information like path cost, distance to goal, and estimated remaining cost. This helps the agent make smarter decisions while searching.
6. Examples of informed search algorithms include Greedy Best First Search and A* Search. These algorithms are widely used in AI applications such as navigation and robotics.

6. Define Greedy Best First Search.

1. Greedy Best First Search is an informed search algorithm that selects the node closest to the goal. It uses heuristic values to guide the search process.
2. The algorithm uses the evaluation function $f(n) = h(n)$. Here, $h(n)$ represents the estimated cost from the current node to the goal node.
3. At each step, the node with the smallest heuristic value is expanded first. This helps the algorithm move toward the most promising path quickly.
4. Greedy Best First Search is usually implemented using a priority queue. Nodes with lower heuristic values are given higher priority for expansion.
5. One advantage of this algorithm is that it expands fewer nodes compared to BFS and DFS. However, it is not guaranteed to find the shortest path.
6. The algorithm may get stuck in loops or explore deep unnecessary paths if the heuristic is poor. Its time and space complexity are both $O(bm)$.

7. What is the evaluation function in A* search?

1. The evaluation function in A* search is used to determine the best node for expansion. It combines both actual cost and estimated future cost.
2. The evaluation function is represented as $f(n) = g(n) + h(n)$. This formula helps A* search find the shortest and most efficient path.
3. In the formula, $g(n)$ represents the actual cost from the start node to the current node. It shows how much cost has already been spent.
4. The term $h(n)$ represents the estimated cost from the current node to the goal node. It is calculated using a heuristic function.
5. By combining $g(n)$ and $h(n)$, A* search balances explored cost and future estimated cost. This makes the algorithm more efficient and accurate.
6. The evaluation function helps A* search remain complete and optimal when the heuristic is admissible. It is widely used in pathfinding and route optimization problems.

8. Define Hill Climbing algorithm.

1. Hill Climbing is a local search algorithm used for solving optimization problems. It continuously moves toward a better neighbouring state.
2. The algorithm selects the best successor based on an objective function. It keeps improving the current state step by step.
3. Hill Climbing moves in the direction of increasing value, similar to climbing a hill. It stops when no neighbouring state has a better value.
4. This algorithm does not maintain a search tree or path information. It only stores the current state and its objective value.
5. Hill Climbing is also called greedy local search because it chooses the best immediate option. It often reaches a solution quickly with low memory usage.
6. One disadvantage is that it may get stuck at local maxima or plateaus. In such situations, it may fail to reach the global optimum solution.

9. What is Simulated Annealing?

1. Simulated Annealing is a local search and optimization algorithm inspired by the annealing process in metallurgy. It is used to find near-optimal solutions for complex problems.
2. The algorithm starts with an initial solution and explores neighbouring solutions randomly. It accepts better solutions and sometimes also accepts worse solutions.
3. Accepting worse solutions helps the algorithm escape local maxima and local minima. This increases the chance of reaching a global optimum solution.
4. Simulated Annealing uses a temperature parameter to control the search process. At high temperature, the algorithm accepts more random moves, while at low temperature it becomes more selective.
5. As the temperature gradually decreases, the search becomes more stable and focused. This cooling process continues until the algorithm reaches a stopping condition.
6. Simulated Annealing is widely used in optimization problems such as scheduling, routing, and artificial intelligence. It is effective for large search spaces where exact solutions are difficult to find.

10. Define Genetic Algorithm.

1. Genetic Algorithm is a search and optimization technique based on the process of natural selection. It uses concepts inspired by biological evolution.
2. In this algorithm, possible solutions are represented as chromosomes or individuals. A group of these individuals is called a population.
3. The algorithm evaluates each solution using a fitness function. Better solutions are selected for producing the next generation.
4. Genetic Algorithm uses operations such as selection, crossover, and mutation. These operations help create new and improved solutions over time.
5. The process continues for several generations until a good or optimal solution is found. It is useful for solving complex optimization and search problems.
6. Genetic Algorithms are widely applied in machine learning, scheduling, robotics, and engineering design. They are effective for problems with very large search spaces.

11. What is Adversarial Search?

1. Adversarial Search is a search technique used in situations where multiple agents compete against each other. It is mainly used in game-playing artificial intelligence.
2. In adversarial search, one player tries to maximize its advantage while the opponent tries to minimize it. The decisions of one player affect the other player's outcome.
3. This type of search is commonly used in games such as chess, tic-tac-toe, and checkers. AI systems use it to decide the best possible move during gameplay.
4. Adversarial search algorithms explore different possible game states and predict future moves. They evaluate which moves will lead to the best result.
5. The Minimax algorithm is one of the most common adversarial search methods. It assumes that both players play optimally to achieve their goals.
6. Adversarial search helps AI systems make strategic decisions in competitive environments. It improves the ability of machines to handle decision-making in games and similar situations.

(5- marks)

1. Explain Breadth First Search (BFS) with example.

1. Breadth First Search (BFS) is a search algorithm used for traversing trees and graphs level by level. It starts searching from the root node and explores all neighbour nodes before moving deeper.
2. BFS works in a breadthwise manner, which means it first completes one level completely. After finishing the current level, it moves to the next level of nodes.
3. The algorithm uses a FIFO queue data structure for storing nodes during traversal. The node inserted first into the queue is expanded first.
4. BFS is complete because it can find a solution if one exists at a finite depth. It is also optimal when path costs increase with depth.
5. One advantage of BFS is that it finds the shortest path with minimum number of steps. Its disadvantage is high memory usage because all nodes at each level must be stored.
6. Example: In a tree starting from node S, BFS visits nodes as $S \rightarrow A \rightarrow B \rightarrow C \rightarrow D \rightarrow G$. It explores all nodes level by level until the goal node is reached.

2. Describe Depth First Search (DFS) with advantages and disadvantages.

1. Depth First Search (DFS) is a search algorithm used for traversing trees and graphs deeply. It starts from the root node and explores one branch completely before backtracking.
2. DFS follows the path to the deepest node first and then moves to another branch. It uses a stack or recursion for implementation.
3. The algorithm works using the LIFO principle, where the most recently added node is expanded first. This helps DFS move deeply into the search space.
4. One advantage of DFS is that it requires less memory compared to BFS. It also reaches the goal quickly if the correct path is explored first.
5. The disadvantage of DFS is that it may enter an infinite loop in large or cyclic graphs. It also does not always guarantee the shortest or optimal solution.
6. Example: Starting from node S, DFS may follow the path $S \rightarrow A \rightarrow B \rightarrow D \rightarrow E$ before backtracking. After backtracking, it continues exploring other nodes until the goal is found.

3. Compare DFS and Depth Limited Search.

1. Depth First Search (DFS) explores nodes deeply without any fixed depth restriction. Depth Limited Search (DLS) is similar to DFS but stops searching after reaching a predefined depth limit.
2. DFS may continue infinitely if the search space contains loops or infinite paths. DLS avoids this problem by restricting the maximum search depth.
3. DFS explores all possible nodes until a solution is found or all paths are exhausted. DLS only explores nodes up to the specified depth limit.
4. DFS does not require a limit parameter for operation. DLS requires a depth limit value to control the search process.
5. Both DFS and DLS are memory efficient because they store only the current path. However, DLS may miss the solution if the goal lies beyond the depth limit.
6. DFS is not always complete in infinite search spaces because it may get trapped in deep paths. DLS becomes complete only if the solution exists within the specified depth limit.
7. The time complexity of DFS is $O(bm)$, while the time complexity of DLS is $O(b\ell)$. Here, m is maximum depth and ℓ is the depth limit.

4. Explain Iterative Deepening Depth First Search (DFID).

1. Iterative Deepening Depth First Search (DFID) is a search algorithm that combines the features of BFS and DFS. It performs DFS repeatedly with increasing depth limits.
2. The algorithm begins with a small depth limit and gradually increases it step by step. This process continues until the goal node is found.
3. DFID combines the fast searching ability of BFS with the memory efficiency of DFS. It is especially useful when the depth of the goal node is unknown.
4. The algorithm is complete if the branching factor is finite. It can also be optimal when path costs do not decrease with depth.
5. One advantage of DFID is that it uses less memory while still finding shallow solutions efficiently. It works well for large search spaces in artificial intelligence problems.
6. The main disadvantage of DFID is repeated searching of previously explored nodes in every iteration. Because of this repeated work, additional processing time is required.
7. The time complexity of DFID is $O(b^d)$, and it is widely used in game searching and AI problem solving.

5. Describe Greedy Best First Search with example.

1. Greedy Best First Search is an informed search algorithm that selects the node closest to the goal. It uses heuristic information to guide the search process efficiently.
2. The algorithm uses the evaluation function $f(n) = h(n)$, where $h(n)$ is the estimated cost to reach the goal. The node with the smallest heuristic value is expanded first.
3. Greedy Best First Search uses a priority queue to store nodes during traversal. Nodes having lower heuristic values are given higher priority for expansion.
4. One advantage of this algorithm is that it expands fewer nodes compared to uninformed search methods. This makes the search process faster in many cases.
5. The algorithm is not always optimal because it only considers heuristic values and ignores actual path cost. It may also get trapped in loops if repeated states are not handled properly.

6. Example: Starting from node S, the algorithm selects node B because it has a lower heuristic value than A. It then follows the path $S \rightarrow B \rightarrow F \rightarrow G$ and reaches the goal node G.
7. The time and space complexity of Greedy Best First Search are both $O(bm)$, where b is branching factor and m is maximum depth. It is widely used in AI pathfinding problems.

6. Explain A* search algorithm with evaluation function.

1. A* search algorithm is an informed search technique used to find the shortest path efficiently. It combines the features of Uniform Cost Search and Greedy Best First Search.
2. The algorithm uses an evaluation function represented as $f(n) = g(n) + h(n)$. This function helps determine the most promising node for expansion.
3. In the evaluation function, $g(n)$ represents the actual cost from the start node to the current node. The term $h(n)$ represents the estimated cost from the current node to the goal.
4. A* expands the node with the lowest $f(n)$ value first during the search process. This helps balance both explored cost and estimated future cost.
5. One advantage of A* search is that it is complete and optimal when the heuristic is admissible. It can solve complex pathfinding problems efficiently.
6. The disadvantage of A* is high memory and time requirements for very large search spaces. Its performance also depends heavily on the quality of the heuristic function.
7. Example: In the given graph, A* selects nodes with the smallest $f(n)$ values and finds the optimal path $S \rightarrow A \rightarrow C \rightarrow G$ with total cost 6.

7. Explain Hill Climbing algorithm and its limitations.

1. Hill Climbing is a local search algorithm used for solving optimization problems. It continuously moves from the current state to a better neighbouring state.
2. The algorithm selects the best successor according to an objective function value. It keeps improving the solution until no better neighbour exists.
3. Hill Climbing does not maintain paths or search trees during execution. It only stores the current state and its objective value, making it memory efficient.
4. This algorithm is also called greedy local search because it chooses the best immediate option without looking ahead. It often reaches a solution quickly in large search spaces.

5. One limitation of Hill Climbing is getting stuck at a local maximum, where neighbouring states are worse but the solution is not globally best. In such cases, the algorithm cannot progress further.
6. Another limitation is the plateau problem, where neighbouring states have equal values and the search loses direction. Random sideways moves may help temporarily but can also lead to endless loops.
7. Hill Climbing may fail to find the global optimum because it only considers immediate neighbours. Despite these limitations, it is widely used in optimization and AI applications.

8. Describe Simulated Annealing with working principle.

1. Simulated Annealing is a local search and optimization algorithm inspired by the annealing process in metallurgy. It is used to find near-optimal solutions for complex problems with large search spaces.
2. The algorithm starts with an initial solution and checks its neighbouring solutions repeatedly. It moves toward better solutions to improve the current state step by step.
3. Unlike Hill Climbing, Simulated Annealing can also accept worse solutions temporarily. This helps the algorithm escape local maxima and avoid getting stuck early.
4. The algorithm uses a temperature parameter to control the acceptance of solutions. At high temperature, random moves are accepted more frequently, while at low temperature the search becomes more stable.
5. During execution, the temperature is gradually reduced in a process called cooling. As cooling continues, the algorithm slowly focuses on the best possible solutions.
6. The working principle involves generating a neighbour, comparing its cost, and deciding whether to accept it based on temperature and probability. This process repeats until the stopping condition is reached.
7. Simulated Annealing is widely used in scheduling, routing, and optimization problems in AI. It is effective for finding good solutions where exact methods are difficult to apply.

9. Explain Minimax algorithm with example.

1. Minimax is an adversarial search algorithm used in two-player games such as chess and tic-tac-toe. It helps the AI choose the best possible move by considering all possible future moves.
2. The algorithm works on the idea that one player tries to maximize the score while the opponent tries to minimize it. Because of this, it is called the Minimax algorithm.
3. Minimax explores the game tree recursively and evaluates terminal states using utility values. The maximizing player selects the highest value, while the minimizing player selects the lowest value.
4. The algorithm assumes that both players play optimally during the game. This allows the AI to make intelligent and strategic decisions.
5. Example: In tic-tac-toe, if one move leads to a win with utility +1 and another leads to a draw with utility 0, the maximizing player selects the winning move. The minimizing player tries to force the lowest possible outcome for the opponent.
6. One advantage of Minimax is that it provides optimal decisions in competitive environments. However, it becomes slow for large game trees because many possible moves must be explored.

10. Describe Alpha-Beta Pruning and its advantages.

1. Alpha-Beta Pruning is an optimization technique used with the Minimax algorithm. It reduces the number of nodes explored in a game tree during search.
2. The algorithm uses two values called alpha and beta for pruning unnecessary branches. Alpha represents the best value for the maximizing player, while beta represents the best value for the minimizing player.
3. During traversal, if a branch cannot improve the current decision, it is ignored or pruned. This reduces unnecessary calculations and improves efficiency.
4. Alpha-Beta Pruning produces the same result as the normal Minimax algorithm. However, it does so by exploring fewer nodes in the search tree.
5. One major advantage is that it significantly reduces computation time in large games. This allows AI systems to search deeper levels within limited time.
6. Another advantage is improved memory and processing efficiency compared to basic Minimax. It makes game-playing AI faster and more practical for real-world applications.
7. Alpha-Beta Pruning is commonly used in chess, checkers, and other strategy games. It improves the performance of adversarial search algorithms in artificial intelligence.

(10 – marks)

1. Explain Genetic Algorithm in detail with operators such as selection, crossover, and mutation.

1. Genetic Algorithm is a search and optimization technique based on the principle of natural evolution. It uses concepts such as survival of the fittest and genetic inheritance to solve problems.
2. In this algorithm, each possible solution is represented as a chromosome or individual. A collection of these individuals forms a population used during the search process.
3. The algorithm begins with a randomly generated population of solutions. Each solution is evaluated using a fitness function to measure its quality.
4. Selection is the first important operator in Genetic Algorithm. In this process, the best individuals with higher fitness values are selected for reproduction.
5. Crossover is the process of combining two parent chromosomes to produce new offspring. This operator helps generate better solutions by mixing useful characteristics of parents.
6. Mutation is another important operator that randomly changes some genes in a chromosome. It helps maintain diversity in the population and avoids premature convergence.
7. After applying selection, crossover, and mutation, a new generation of solutions is created. The algorithm continues this process repeatedly for several generations.
8. Genetic Algorithm gradually improves the quality of solutions with every iteration. It stops when an optimal or near-optimal solution is obtained.
9. Genetic Algorithms are widely used in machine learning, robotics, scheduling, and optimization problems. They are useful for solving complex problems where traditional methods are difficult to apply.
10. One major advantage of Genetic Algorithm is its ability to handle very large search spaces effectively. However, it may require high computation time for complex problems.

2. What is Adversarial Search? Explain Minimax algorithm with game tree example.

1. Adversarial Search is a search technique used in competitive environments where two or more players oppose each other. It is mainly used in artificial intelligence for game-playing systems.
2. In adversarial search, one player tries to maximize its benefit while the opponent tries to minimize it. The decision of one player directly affects the outcome of the other player.
3. The Minimax algorithm is the most common adversarial search algorithm. It helps the AI select the best move by exploring all possible future moves.
4. Minimax works on two types of players called Maximizer and Minimizer. The Maximizer tries to obtain the highest score, while the Minimizer tries to reduce the score.
5. The algorithm builds a game tree containing all possible moves and outcomes. Terminal states are evaluated using utility values such as win, loss, or draw.
6. During evaluation, the Maximizer selects the highest utility value from its child nodes. The Minimizer selects the lowest utility value to reduce the opponent's advantage.
7. Example: In a tic-tac-toe game tree, suppose one branch gives utility +1 for a win and another gives 0 for a draw. The Maximizer selects the branch with utility +1 as the best move.
8. Minimax assumes that both players always play optimally and intelligently. This allows the algorithm to make strategic and accurate decisions.
9. One advantage of Minimax is that it provides the best possible move in two-player games. However, it becomes computationally expensive for very large game trees.
10. Minimax is widely used in games such as chess, checkers, and tic-tac-toe. It forms the foundation for advanced adversarial search methods like Alpha-Beta Pruning.

3. Explain Alpha-Beta Pruning and show how it improves efficiency of Minimax.

1. Alpha-Beta Pruning is an optimization technique used to improve the efficiency of the Minimax algorithm. It reduces the number of nodes explored in the game tree.
2. The algorithm uses two parameters called alpha and beta during the search process. Alpha represents the best value for the maximizing player, while beta represents the best value for the minimizing player.
3. During traversal, if a branch cannot provide a better result than the current best value, that branch is pruned. This means the unnecessary branch is ignored completely.
4. Alpha-Beta Pruning does not change the final decision of the Minimax algorithm. It produces the same optimal result while reducing computations.
5. Example: Suppose the Maximizer already has a value of 8 and another branch can only produce values lower than 8. In this case, the remaining nodes of that branch are pruned because they cannot improve the result.
6. By eliminating unnecessary branches, the algorithm searches fewer nodes in the game tree. This increases the speed and efficiency of adversarial search.
7. One major advantage of Alpha-Beta Pruning is reduced computation time for large games. It allows AI systems to search deeper levels within limited time.
8. Another advantage is lower memory and processor usage compared to basic Minimax. This makes it more practical for real-time applications and complex games.
9. Alpha-Beta Pruning is commonly used in games such as chess, checkers, and strategic board games. It helps AI systems make faster and more intelligent decisions.
10. The efficiency of Minimax improves greatly with Alpha-Beta Pruning because many unnecessary branches are skipped. As a result, the algorithm becomes faster without affecting accuracy.

4. Solve a game tree using Minimax and Alpha-Beta Pruning.

1. Minimax is an adversarial search algorithm used in two-player games to select the best move. It assumes that one player tries to maximize the score while the opponent tries to minimize it.
2. Consider a simple game tree where the root node is a Maximizer node with two child Minimizer nodes. The terminal values under the left branch are 3 and 5, while the right branch has values 2 and 9.
3. In the left Minimizer branch, the algorithm compares values 3 and 5. Since it is a Minimizer node, it selects the smaller value, which is 3.
4. In the right Minimizer branch, the algorithm compares values 2 and 9. The Minimizer again selects the smaller value, which is 2.
5. Now the root Maximizer node compares the results obtained from both branches. It compares 3 and 2, then selects the maximum value, which is 3.
6. Therefore, according to the Minimax algorithm, the optimal decision is the left branch with utility value 3. This ensures the best possible outcome for the maximizing player.
7. Alpha-Beta Pruning improves this process by removing unnecessary branches from the search tree. It uses alpha and beta values to decide when pruning can occur.
8. Suppose the left branch already provides a value of 3 for the Maximizer. While exploring the right branch, if the Minimizer finds a value of 2, further exploration can stop because the Maximizer already has a better choice.
9. In this case, the node with value 9 does not need to be explored due to pruning. This reduces the number of computations while still producing the same final decision.
10. Thus, Alpha-Beta Pruning increases the efficiency of Minimax by skipping unnecessary branches. It helps AI systems make faster decisions in games such as chess and tic-tac-toe.